

E-Commerce Order & Customer Analytics — SQL Mega Project

Context

You are a Data Analyst hired by an e-commerce company operating across India. The business wants deep visibility into its customers, products, orders, payments, and returns. Your job is to write SQL queries that answer real business questions and build reusable analytical views.

Section 1 — Database Setup & Basics

Concepts: DDL, DML, integrity constraints, data types

1. Write the CREATE TABLE statements for all 8 tables with appropriate data types, primary keys, foreign keys, NOT NULL, and UNIQUE constraints. (The schema is provided — reproduce it from scratch.)
 2. Insert at least 3 new customers using `INSERT INTO`.
 3. Update the `stock_qty` of product_id 15 to 0 since it is discontinued.
 4. Delete the customer with no phone number and no orders.
 5. Use `TRUNCATE` on a test table you create, and explain how it differs from `DELETE`.
 6. Demonstrate `COMMIT` and `ROLLBACK` with a transaction that updates an order status.
-

Section 2 — Filtering & Pattern Matching

Concepts: SELECT, WHERE, LIKE, BETWEEN, DISTINCT, wildcards, aliases

7. List all customers from cities that start with the letter 'K'.
 8. Find all products whose name contains 'air' (case-insensitive).
 9. Get all orders placed between 1st July 2023 and 31st December 2023.
 10. List distinct payment methods used across all transactions.
 11. Find all customers whose email ends with '@email.com' and who have no phone number on record.
 12. Use aliases to display: customer full name as `Customer`, city as `Location`, and signup year as `Member Since`.
-

Section 3 — Operators & Sorting

Concepts: arithmetic, logical, comparison operators, ORDER BY, LIMIT

13. Calculate the profit margin per product: $((\text{price} - \text{cost_price}) / \text{price} * 100)$, aliased as `margin_pct`. Show top 5 most profitable products.
 14. List all orders where discount is greater than 0, sorted by discount descending.
 15. Show all products that are active AND have stock below 100.
 16. Find all payments where amount paid is above ₹50,000 OR method is 'COD'.
-

Section 4 — Aggregations & Grouping

Concepts: COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING

17. Total revenue generated per payment method (only for 'Success' payments).
 18. Count of orders per status. Show only statuses with more than 2 orders.
 19. Average order value per customer. Show only customers whose average exceeds ₹20,000.
 20. Which category has the highest total quantity of products sold? (Join `order_items` → `products` → `categories`.)
 21. Find the month with the highest number of orders placed (extract month from `order_date`).
-

Section 5 — NULL Handling

Concepts: IS NULL, IS NOT NULL, COALESCE, NULLIF

22. List all customers where phone number is NULL.
 23. Display customer names with their phone — if phone is NULL, show 'Not Provided' using `COALESCE`.
 24. Find orders where `coupon_code` is NULL and status is 'Delivered'. How many customers never used a coupon?
 25. Use `NULLIF` to avoid division-by-zero when computing average discount only for orders where `discount_pct > 0`.
-

Section 6 — Joins

Concepts: INNER, LEFT, RIGHT, FULL OUTER, SELF, CROSS JOIN

26. List every order with customer name, city, product name, quantity, and order status. (INNER JOIN across orders, customers, order_items, products.)
 27. Show all customers and their orders. Include customers who have never placed an order. (LEFT JOIN)
 28. Find products that have never appeared in any order. (LEFT JOIN with NULL check)
 29. Self-join: Find pairs of customers from the same city.
 30. Cross join: Generate all combinations of regions and payment methods (for a hypothetical marketing grid).
 31. Full outer join (or simulated): Identify orders with no matching payment record, and payments with no matching order.
-

Section 7 — Date Functions

Concepts: date arithmetic, EXTRACT, DATE_DIFF, CURRENT_DATE, DATE_FORMAT

32. How many days has each customer been a member as of today?
 33. List all orders placed in Q4 (October–December) across any year.
 34. Find customers who signed up more than 2 years ago but have placed fewer than 2 orders total.
 35. Show month-wise order count for 2024 in the format 'YYYY-MM'.
 36. Calculate the number of days between order_date and payment_date for each order.
-

Section 8 — String Functions

Concepts: UPPER, LOWER, CONCAT, LENGTH, SUBSTRING, TRIM, REPLACE

37. Display all customer names in UPPERCASE.
 38. Create a display label combining product name and price: e.g.,

Samsung Galaxy S23 – ₹62999

.
 39. Extract the domain from customer email addresses (everything after '@').
 40. Find all products whose name has more than 20 characters.
 41. Replace 'Delivered' with 'Completed' in the status column for display purposes (do not update the table).
-

Section 9 — CASE Expressions

Concepts: CASE WHEN, ELSE

42. Segment customers by membership duration:
 - < 1 year → 'New'
 - 1-2 years → 'Regular'
 - ≥ 2 years → 'Loyal'
 43. Classify orders by order value (after discount) into 'Low' (<5K), 'Mid' (5K-50K), 'High' (>50K).
 44. For each product, label margin as 'Low Margin' (<20%), 'Healthy' (20-40%), 'Premium' (>40%).
 45. Flag payments as 'Discount Applied' or 'Full Price' based on whether the order had a coupon code.
-

Section 10 — Subqueries & CTEs

Concepts: correlated subqueries, scalar subqueries, IN/EXISTS, WITH clause

46. Find all customers who have placed more orders than the average number of orders per customer. (Subquery)
 47. List products that have never been ordered. (NOT IN / NOT EXISTS — write both versions.)
 48. Using a CTE, calculate each customer's total spend and rank them by spend descending.
 49. Multi-level CTE: First compute monthly revenue, then compute quarter-over-quarter change.
 50. Correlated subquery: For each order, show whether the customer's order value is above or below their own personal average.
-

Section 11 — Window Functions

Concepts: ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, SUM OVER, AVG OVER

51. Rank customers by total spend using `RANK()` and `DENSE_RANK()` — show the difference.
52. Assign row numbers to orders per customer, ordered by order_date. Identify each customer's first order.
53. Running total of revenue month by month using `SUM() OVER (ORDER BY month)`.
54. Use `LAG()` to compute month-over-month revenue change.

55. For each product sold, show the previous product ordered in the same order using `LAG()` `OVER (PARTITION BY order_id)`.
-

Section 12 — Views

Concepts: CREATE VIEW, querying views

56. Create a view `vw_customer_summary` showing: customer name, city, region, total orders, total spend, premium status.
57. Create a view `vw_product_performance` showing: product name, category, total units sold, total revenue, profit.
58. Create a view `vw_monthly_revenue` showing: year, month, total revenue, total orders.
59. Query your views to answer: Which region generated the highest revenue? Which product is most profitable?
-

Section 13 — UNION & UNION ALL

Concepts: UNION, UNION ALL, set operations

60. UNION: List all cities from both the customers table and a manually written list of target cities — remove duplicates.
61. UNION ALL: Combine delivered orders and returned orders into a single activity log with a `event_type` label column.
62. Use UNION to produce a single report of top 3 products by revenue and top 3 products by quantity sold.
-

Section 14 — Normalization (Conceptual + Applied)

Concepts: 1NF, 2NF, 3NF

63. Document in comments/notes: How does this schema satisfy 1NF, 2NF, and 3NF? Identify one example of a potential normalization violation you deliberately avoided.
64. Create an `addresses` table to split city/region out of customers into a proper normalized structure. Write the migration INSERT.
-

Section 15 — Final Business Dashboard Queries

These are the GitHub-showcase queries. Each should be clean, well-commented, and production-quality.

65. **Customer Lifetime Value Report:** Total spend, order count, avg order value, first order date, last order date, membership tier (CASE) — for all customers.
 66. **Product Profitability Report:** Product, category, units sold, revenue, cost, profit, margin%, ranked by profit.
 67. **Regional Sales Breakdown:** Region, total orders, total revenue, top product in that region (subquery).
 68. **Cohort-style Retention:** Group customers by signup year. Show how many placed orders in their first year vs overall.
 69. **Payment Funnel:** Total orders → Successful payments → Failed → Refunded. Show conversion % at each stage.
 70. **Returns Analysis:** Return rate by category. Which category has the highest return rate?
-

Deliverable Format (GitHub Structure)

```
ecommerce-sql-project/
├─ README.md                ← project overview, schema diagram description
├─ schema/
│   └─ create_tables.sql
├─ data/
│   └─ seed_data.sql
├─ queries/
│   ├── 01_basics.sql
│   ├── 02_filtering.sql
│   ├── 03_aggregations.sql
│   ├── 04_nulls.sql
│   ├── 05_joins.sql
│   ├── 06_date_string_functions.sql
│   ├── 07_case_expressions.sql
│   ├── 08_subqueries_ctes.sql
│   ├── 09_window_functions.sql
│   ├── 10_views.sql
│   ├── 11_unions.sql
│   └─ 12_final_dashboard.sql
```

```
└─ normalization/  
   └─ normalization_notes.sql
```

Evaluation Checklist (Self-review before pushing)

- ☐ Every query has a comment explaining what it answers
- ☐ Window functions use both PARTITION BY and ORDER BY where relevant
- ☐ CTEs are named meaningfully (not `cte1`, `cte2`)
- ☐ Views are queried separately to confirm they work
- ☐ Final dashboard queries return clean, readable output
- ☐ README explains the domain, schema, and what each file contains